One keyword decides who can see the base class.

public, protected or private.

```
class A : public    Base {};
class B : protected Base {};
class C : private   Base {};
```

Same class, three very different relationships.

swipe →

What the base's members become

INHERITANCE	BASE PUBLIC	BASE PROTECTED
`public`	**public**	protected
`protected`	protected	protected
`private`	**private**	**private**

The base's `private` members are never accessible from the derived class, whatever you choose.

public: "is a"

```
class Dog : public Animal {};

Dog d;
Animal& a = d;      // ok
d.name();           // ok
```

A `Dog` can be used anywhere an `Animal` is expected.

This is the one you use for polymorphism. Everything else is the exception.

private: "built with"

```
class Stack : private std::vector<int> {
public:
    void push(int x) { push_back(x); }
};

Stack s;
s.push(1);          // ok
s.push_back(2);     // error
```

The base is an implementation detail. Outsiders cannot see it or convert to it.

Often a data member does the same job more clearly.

Who can use `Derived` as a `Base`?

INHERITANCE	WHO CAN CONVERT
`public`	**everyone**
`protected`	the class and its subclasses
`private`	**only the class itself**

Friends count as the class itself.

The short version

USE	WHEN
`public`	it **is a** kind of the base
`private`	it is **built using** the base
`protected`	almost never

With `class` the default is **private**; with `struct` it is **public**.

When did you last use private inheritance?